# Green University of Bangladesh
# Department of Computer Science and Engineering (CSE)

**Faculty of Sciences and Engineering**

**Semester: (Spring , Year:2026), B.Sc. in CSE (Day)**

**LAB REPORT NO: 1**

**Course Title:** Artificial intelligence

**Course Code:** CSE 316 **Section:** 232-D5

Lab Experiment Name: Introduction to Basic Operations on Python.

**Student Details**

| | Name | ID |
|---|---|---|
| **1.** | Rukonuzzaman Topu | 232002280 |

**Submission Date** : **2/14/2026**

**Course Teacher's Name** : Babe Sultana

**[For Teachers use only: Don't Write Anything inside this box]**

**Lab Report Status**

**Marks: ……** **Signature:.....**

**Comments:.....** **Date:.....**

# 1. Introduction:

This lab report focuses on the fundamental operations of Python programming. The primary goal of this lab is to develop a clear understanding of core concepts such as loops, conditional statements, functions, lists, sets, and dictionaries. Through solving various problems, we practice applying logical thinking step by step to produce accurate and efficient results.

## 2. OBJECTIVES/AIM

1. To understand and practice fundamental Python operations such as variables, input/output handling, loops, conditional statements, functions, and built-in data structures like lists, sets, and dictionaries. This helps build a strong foundation in Python programming.

2. To improve logical thinking and problem-solving skills using Python by breaking problems into smaller steps, designing algorithms, and implementing solutions through structured and efficient code. This develops the ability to think analytically and write clear, organized programs.

## 3. PROCEDUR

In this lab, we completed a wide range of problem-solving tasks using Python to strengthen our understanding of basic programming concepts and logical thinking skills.

First, we worked with numbers and loops. To calculate the sum of odd and even numbers, we examined each number individually using the modulus (`%`) operator to determine whether it was divisible by 2. Based on the result, we added the number to either the odd or even total. To find the smallest number in a group, we compared each element one by one and continuously updated a minimum variable whenever a smaller value was found.

We also selected numbers between 50 and 100 that were divisible by 3 but not divisible by 5. This was done using conditional statements with logical operators, and the qualifying numbers were added to produce a final sum. To determine the second-highest number in a list, we maintained two separate variables to store the highest and second-highest values, updating them appropriately during iteration.

For mathematical operations, we calculated factorials by multiplying numbers sequentially from 1 up to n using a loop. Similarly, we generated the Fibonacci series by keeping track of the two previous numbers and updating them to produce the next term in the sequence.

We practiced using functions by creating one to determine the largest number between two inputs using conditional statements. Another function was written to calculate the sum of given parameters

and return the result. This helped us understand how functions improve code reusability and organization.

In list-based problems, we calculated the total sum of elements by iterating through the list and accumulating their values. To find the largest number in a list, we compared each element and updated the maximum value accordingly. We also counted how many numbers in a list were positive, negative, or zero using conditional checks.

Prime number verification was performed by checking divisibility from 2 up to n−1 to determine whether a number had any factors other than 1 and itself. We printed numbers divisible by both 4 and 6 by applying multiple conditions together. To find the maximum of three numbers, we compared their values step by step using conditional logic.

We identified whether a number was even or odd using the modulus operator. Duplicate elements in a list were removed by creating a new list and adding only those values that were not already present. The average of list elements was calculated by dividing the total sum by the total count of elements. Additionally, we determined the difference between the largest and smallest numbers by subtracting the minimum value from the maximum value.

Finally, we worked with dictionaries. We identified the key associated with the maximum value by comparing all values and tracking the highest one. We also merged two dictionaries by combining their key-value pairs into a single dictionary structure.

Overall, these exercises helped us apply loops, conditionals, functions, lists, sets, and dictionaries effectively, improving both our coding skills and logical reasoning abilities in Python.

## 4.Implementation:

**Code :**

**<u>1st:</u>**

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

evenSum = 0

oddSum = 0

for num in numbers:

    if num % 2 == 0:

        evenSum += num

    else:
```

```
        oddSum += num
print("Sum of even numbers:", evenSum)
print("Sum of odd numbers:", oddSum)
```

**2nd:**

```
numbers = [10, 8, 9, 7, 5, 6]
smallest = numbers[0]
for num in numbers:
    if num < smallest:
        smallest = num
print("The smallest number is:", smallest)
```

**3rd:**

```
totalSum = 0
for num in range(50, 101):
    if num % 3 == 0 and num % 5 != 0:
        totalSum += num
print("The sum of numbers between 50 and 100 that are divisible by 3 and not divisible by 5 is:", totalSum)
```

**4th:**

```
numbers = [10, 8, 9, 7, 5, 6]
largest = float('-inf')
second_largest = float('-inf')
for num in numbers:
```

```
    if num > largest:
        second_largest = largest
        largest = num
    elif largest > num > second_largest:
        second_largest = num
print("The second-highest number is:", second_largest)
```

**5th:**

```
number = 5
factorial = 1
for i in range(1, number + 1):
    factorial *= i
print(f"The factorial of {number} is: {factorial}")
```

**6th:**

```
def fibonacci(n):
    fib_series = []
    a, b = 0, 1
    for _ in range(n):
        fib_series.append(a)
        a, b = b, a + b
    return fib_series
n = 10
```

```
print(f"The first {n} numbers in the Fibonacci series are: {fibonacci(n)}")
```

**7th:**

```
def find_largest(num1, num2):
    if num1 > num2:
        return num1
    elif num2 > num1:
        return num2
    else:
        return "Both numbers are equal"
number1 = 10
number2 = 20
largest = find_largest(number1, number2)
print(f"The largest number between {number1} and {number2} is: {largest}")
```

**8th:**

```
def sum_of_numbers(*args):
    total_sum = sum(args)
    return total_sum
numbers = [1, 2, 3, 4, 5]
result = sum_of_numbers(*numbers)
print(f"The sum of the numbers {numbers} is: {result}")
```

**9th:**

```
numbers = [1, 2, 3, 4, 5]
total_sum = sum(numbers)
print(f"The sum of the numbers {numbers} is: {total_sum}")
```

**10th:**

```
numbers = [10, 8, 9, 7, 5, 6]
largest = float('-inf')
for num in numbers:
    if num > largest:
        largest = num
print("The largest number is:", largest)
```

**11Th:**

```
numbers = [1, -2, 3, 0, -5, 6]
positive_count = 0
negative_count = 0
for num in numbers:
    if num > 0:
        positive_count += 1
    elif num < 0:
        negative_count += 1
print(f"Positive numbers: {positive_count}")
print(f"Negative numbers: {negative_count}")
zero_count = len(numbers) - positive_count - negative_count
print(f"Zeroes: {zero_count}")
```

**12Th:**

```
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True
number = 29
if is_prime(number):
    print(f"{number} is a prime number.")
else:    print(f"{number} is not a prime number.")
```

**13th:**

```
for num in range(1, 101):
    if num % 4 == 0 and num % 6 == 0:
        print(num)
```

**14TH:**

```
def max_of_three(a, b, c):
    return max(a, b, c)
```

```
num1 = 10

num2 = 20

num3 = 15

result = max_of_three(num1, num2, num3)

print(f"The maximum of the numbers {num1}, {num2}, and {num3} is: {result}")
```

**15th:**

```
def check_even_odd(num):

    if num % 2 == 0:

        return "Even"

    else:

        return "Odd"

number = 10

result = check_even_odd(number)

print(f"The number {number} is {result}.")
```

**16th:**

```
numbers = [1, 2, 3, 4, 5, 2, 3, 6]

unique_numbers = list(set(numbers))

print(f"The unique numbers from the list {numbers} are: {unique_numbers}")
```

**17th:**

```
numbers = [10, 20, 30, 40, 50]

average = sum(numbers) / len(numbers)

print(f"The average of the numbers {numbers} is: {average}")
```

**18th:**

```
numbers = [10, 8, 9, 7, 5, 6]

largest = float('-inf')

smallest = float('inf')

for num in numbers:

    if num > largest:

        largest = num

    if num < smallest:

        smallest = num

difference = largest - smallest

print("The difference between the largest and smallest numbers is:", difference)
```

**19th:**

```
my_dict = {'a': 10, 'b': 20, 'c': 15}

max_key = max(my_dict, key=my_dict.get)

print(f"The key with the maximum value in the dictionary {my_dict} is: '{max_key}' with a value of {my_dict[max_key]}.")
```

**20th:**

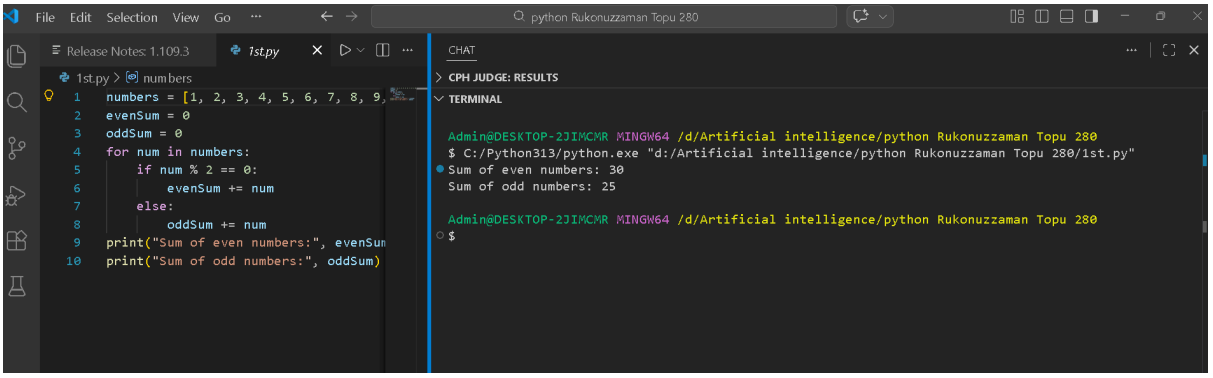
```
dict1 = {'a': 1, 'b': 2}

dict2 = {'c': 3, 'd': 4}
```

merged_dict = {**dict1, **dict2}

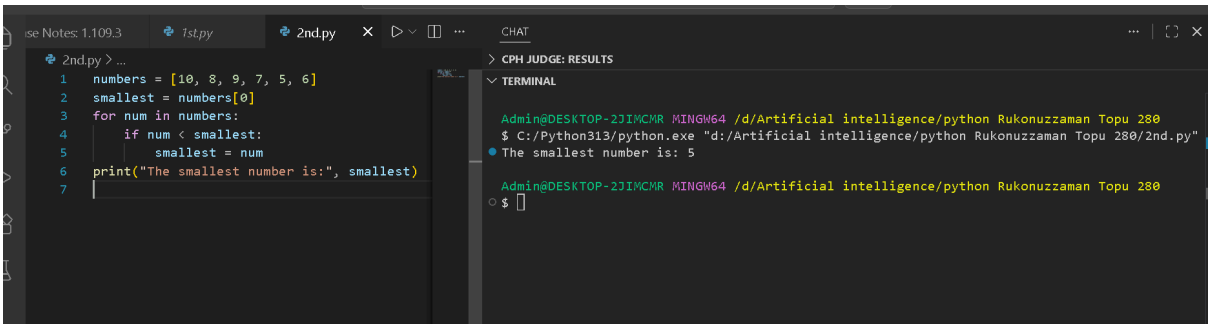
print(f"The merged dictionary is: {merged_dict}")

**Output :**

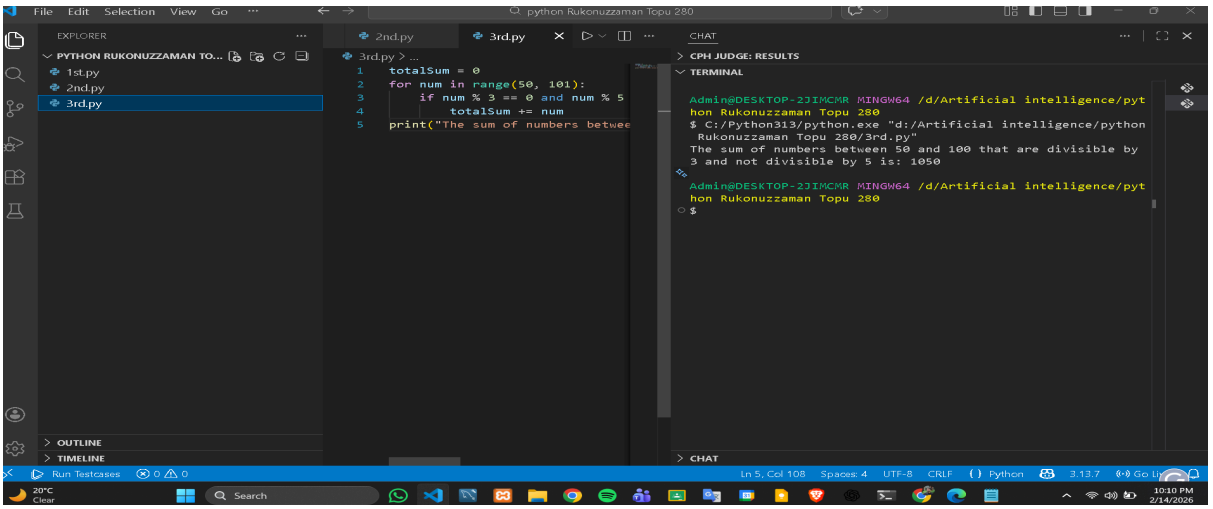
**1st:**



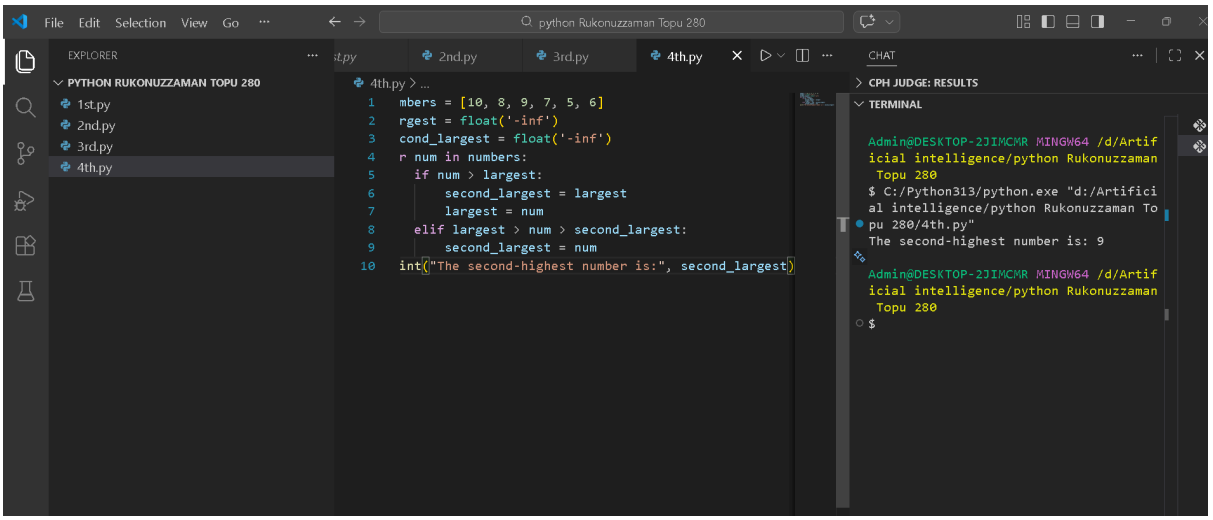

**2nd:**



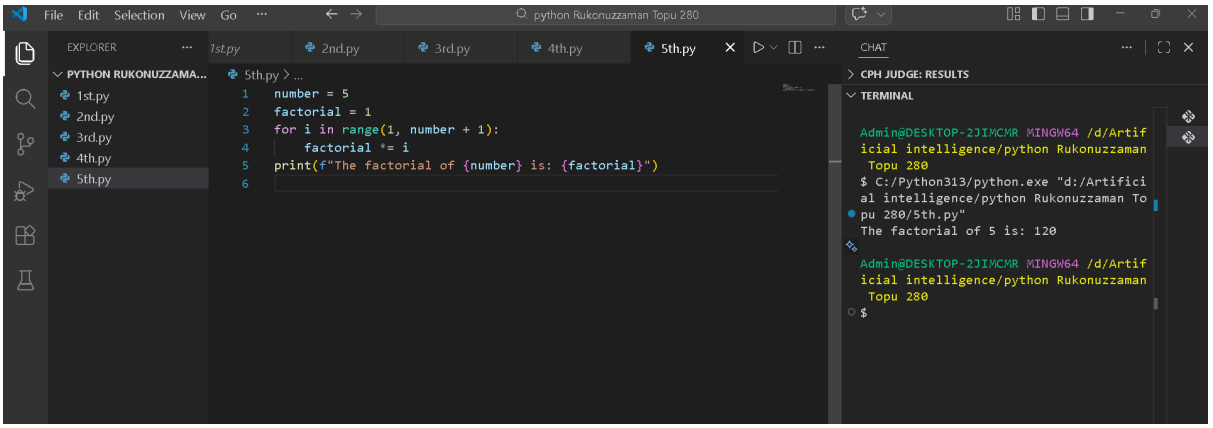

**3rd:**



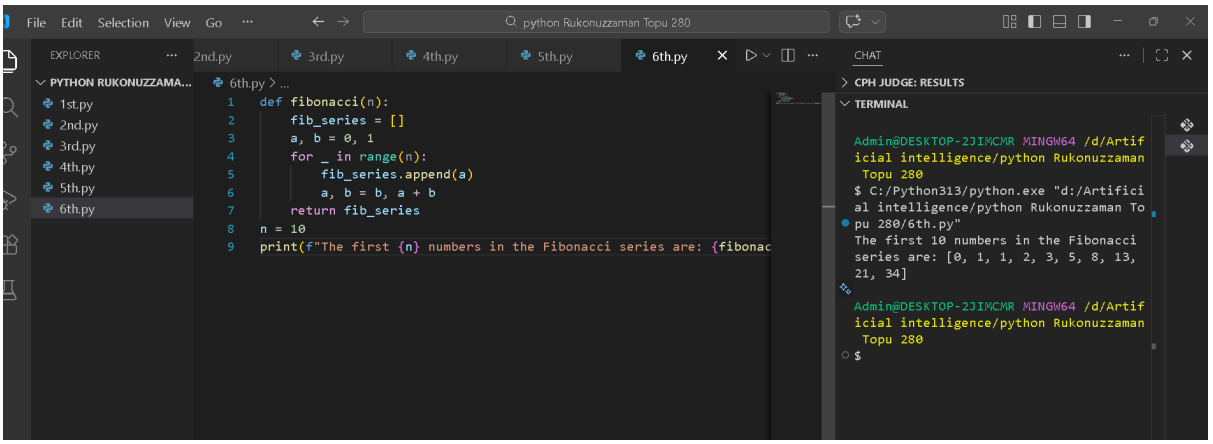
## 4th:


File Edit Selection View Go
python Rukonuzzaman Topu 280
EXPLORER
PYTHON RUKONUZZAMAN TOPU 280
1st.py
2nd.py
3rd.py
4th.py
4th.py > ...
mbers = [10, 8, 9, 7, 5, 6]
rgest = float('-inf')
cond_largest = float('-inf')
r num in numbers:
if num > largest:
second_largest = largest
largest = num
elif largest > num > second_largest:
second_largest = num
int("The second-highest number is:", second_largest)
CHAT
CPH JUDGE: RESULTS
TERMINAL
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/4th.py"
The second-highest number is: 9
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$

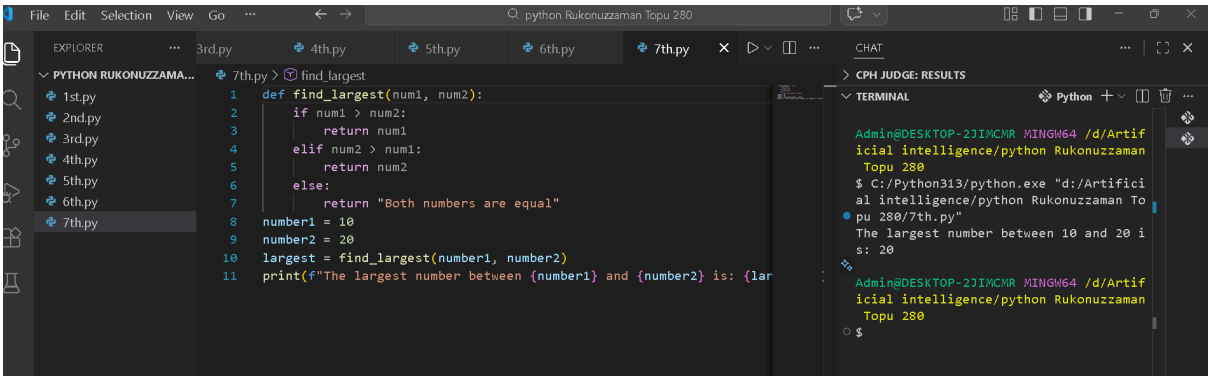
## 5th:


File Edit Selection View Go
python Rukonuzzaman Topu 280
EXPLORER
PYTHON RUKONUZZAMA...
1st.py
2nd.py
3rd.py
4th.py
5th.py
5th.py > ...
number = 5
factorial = 1
for i in range(1, number + 1):
factorial *= i
print(f"The factorial of {number} is: {factorial}")
CHAT
CPH JUDGE: RESULTS
TERMINAL
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/5th.py"
The factorial of 5 is: 120
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$

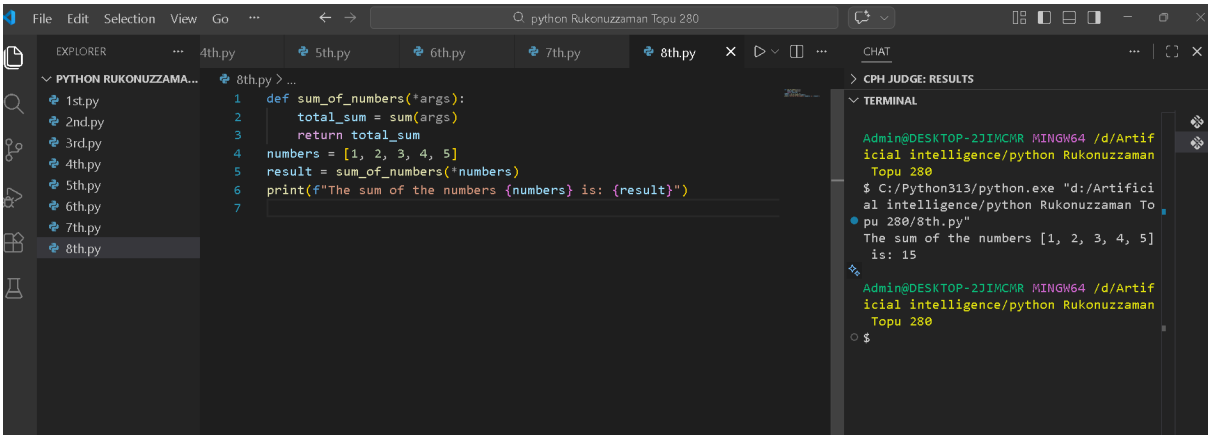
## 6th:


File Edit Selection View Go
python Rukonuzzaman Topu 280
EXPLORER
PYTHON RUKONUZZAMA...
1st.py
2nd.py
3rd.py
4th.py
5th.py
6th.py
6th.py > ...
def fibonacci(n):
fib_series = []
a, b = 0, 1
for _ in range(n):
fib_series.append(a)
a, b = b, a + b
return fib_series
n = 10
print(f"The first {n} numbers in the Fibonacci series are: {fibonac
CHAT
CPH JUDGE: RESULTS
TERMINAL
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/6th.py"
The first 10 numbers in the Fibonacci series are: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$

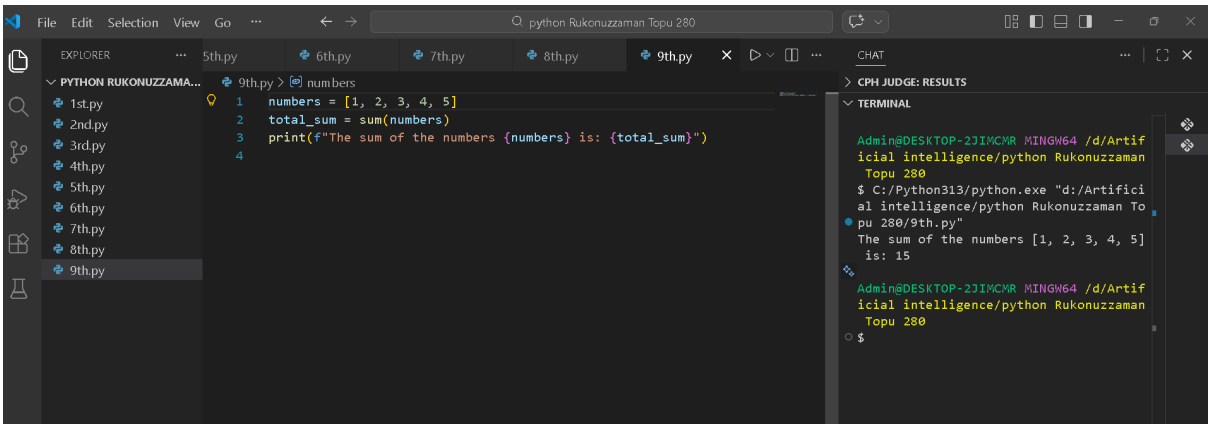
## 7th:



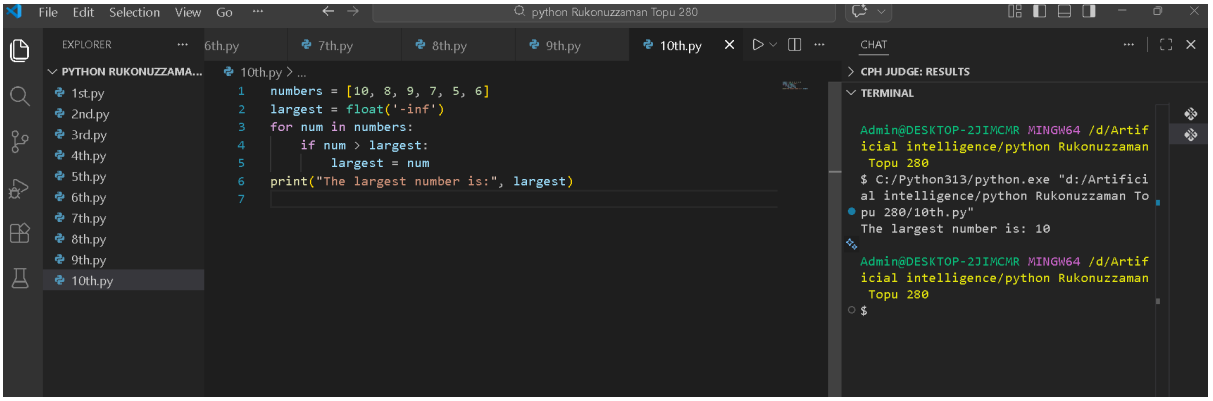

## 8th:



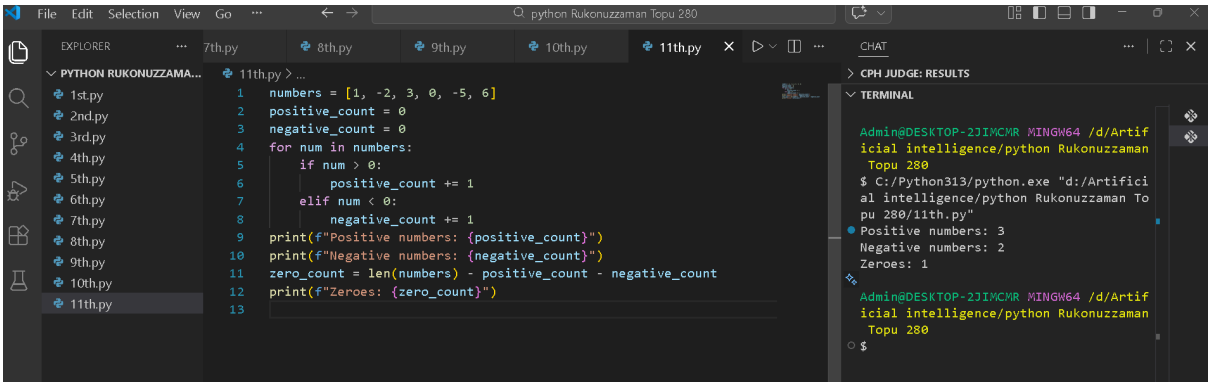

## 9th:



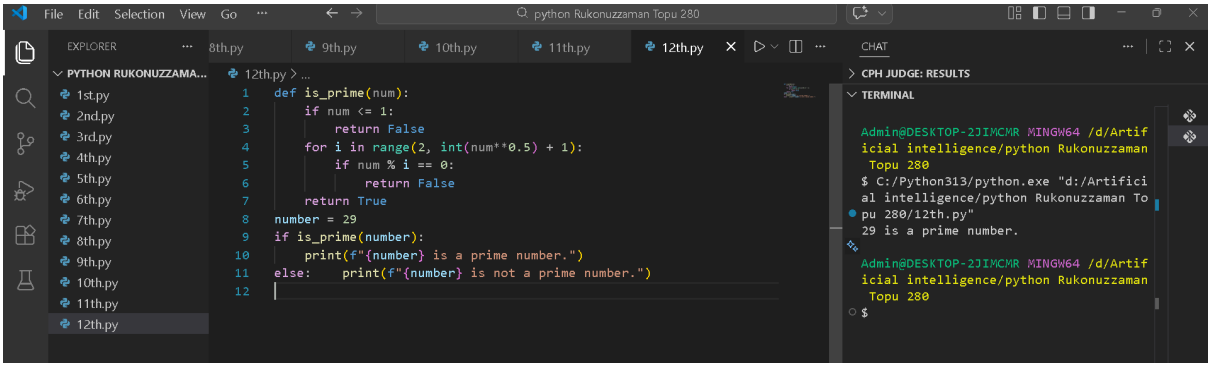

## 10th:



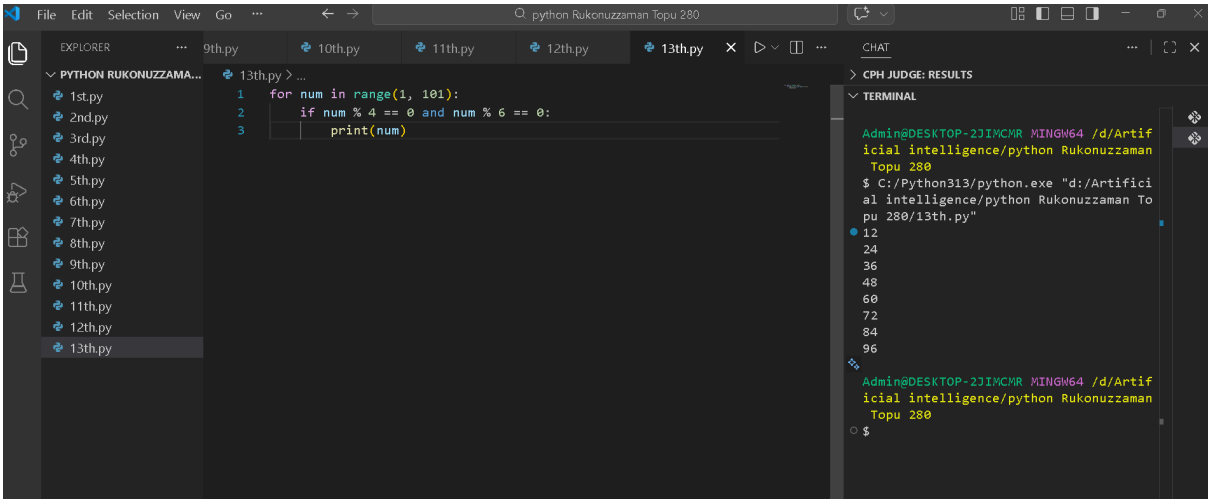

## 11th:



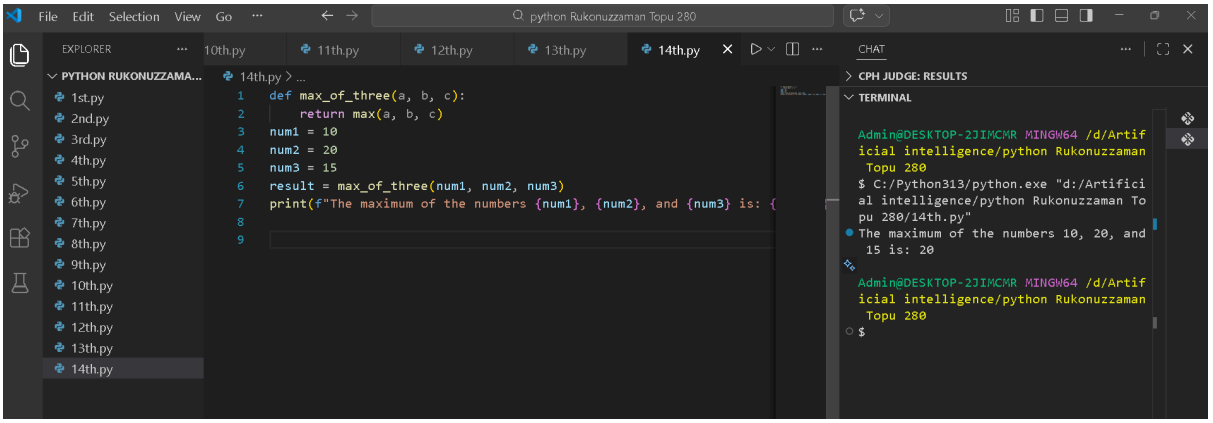

## 12th:



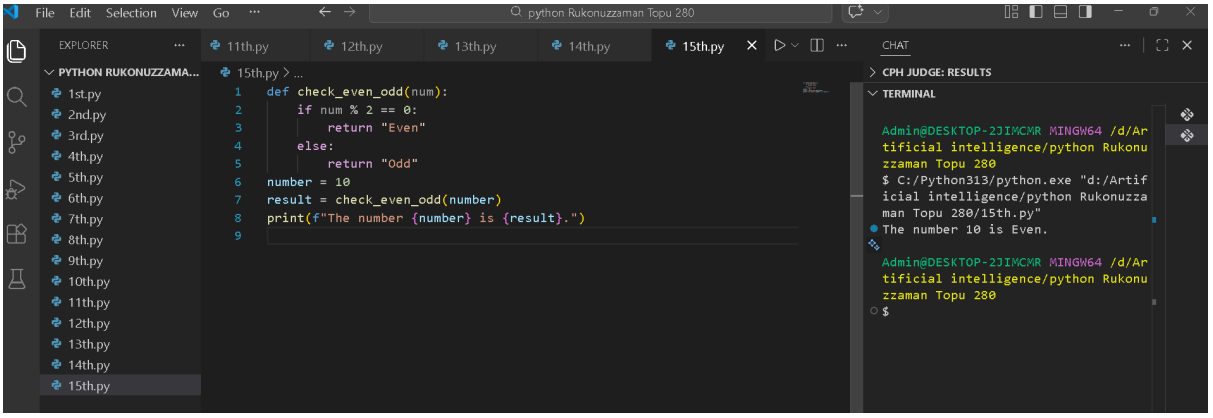

## 13th:




## 14th:




## 15th:




## 16th:

```python
numbers = [1, 2, 3, 4, 5, 2, 3, 6]
unique_numbers = list(set(numbers))
print(f"The unique numbers from the list {numbers} are: {unique_number
```

```
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/16th.py"
The unique numbers from the list [1, 2, 3, 4, 5, 2, 3, 6] are: [1, 2, 3, 4, 5, 6]
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$
```

## 17th:

```python
numbers = [10, 20, 30, 40, 50]
average = sum(numbers) / len(numbers)
print(f"The average of the numbers {numbers} is: {average}")
```

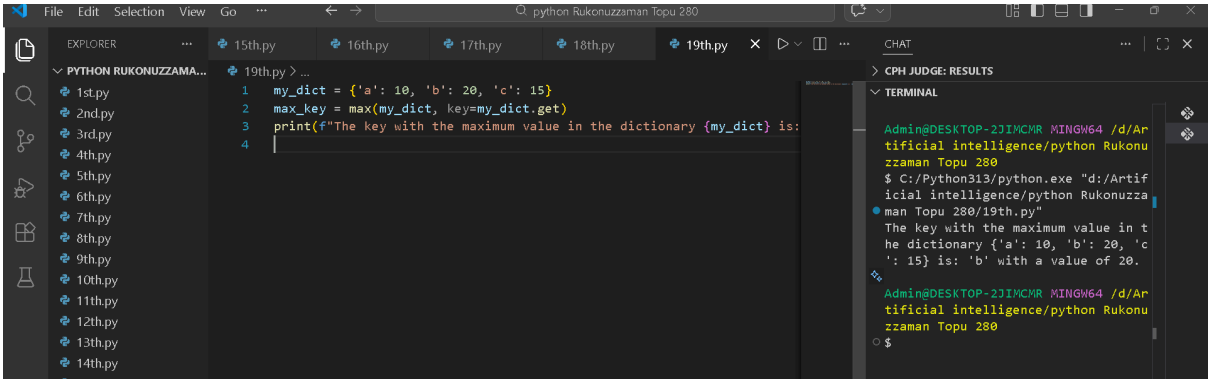
```
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/17th.py"
The average of the numbers [10, 20, 30, 40, 50] is: 30.0
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
```

## 18th:

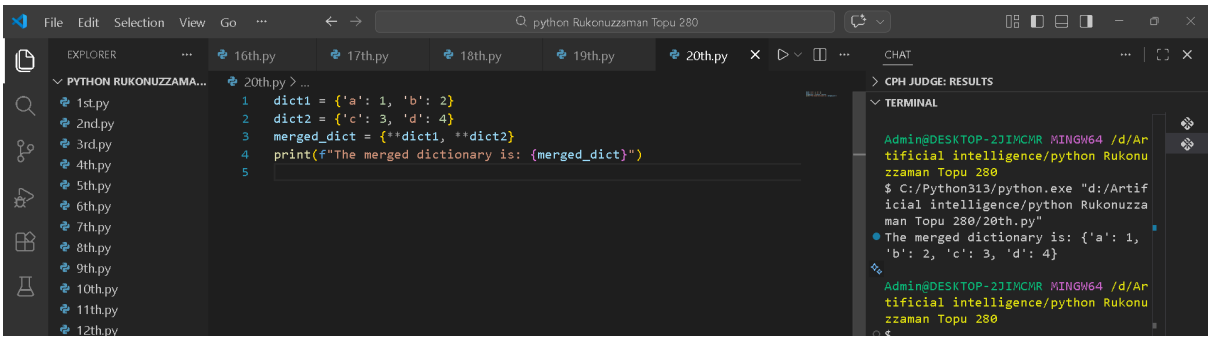
```python
numbers = [10, 8, 9, 7, 5, 6]
largest = float('-inf')
smallest = float('inf')
for num in numbers:
    if num > largest:
        largest = num
    if num < smallest:
        smallest = num
difference = largest - smallest
print("The difference between the largest and smallest numbers is:", d
```

```
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$ C:/Python313/python.exe "d:/Artificial intelligence/python Rukonuzzaman Topu 280/18th.py"
The difference between the largest and smallest numbers is: 5
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intelligence/python Rukonuzzaman Topu 280
$
```

## 19th:




**20th:**




## 6. ANALYSIS AND DISCUSSION:

In this lab, we explored and practiced fundamental concepts of Python programming through a variety of problem-solving exercises. The tasks were designed to strengthen our understanding of basic programming structures such as loops, conditional statements, functions, lists, sets, and dictionaries. By implementing these concepts in practical problems, we were able to clearly understand how each structure works and how they can be combined to build complete solutions.

One important aspect of this lab was developing logical thinking. For example, while calculating sums, finding minimum and maximum values, or determining prime numbers, we had to carefully design step-by-step logic before writing the code. This improved our ability to analyze problems and break them into smaller, manageable steps.

Working with loops helped us understand iteration and repetitive execution, while conditional statements allowed us to control program flow based on specific conditions. Functions helped organize the code into reusable blocks, making programs more structured and readable. Lists and dictionaries enhanced our knowledge of data handling and storage, while removing duplicates and merging dictionaries gave us practical experience in data manipulation.

Overall, this lab was very helpful in strengthening our foundational knowledge of Python. It improved our coding efficiency, logical reasoning, and confidence in solving programming

problems. The skills gained from this lab will be essential for learning advanced programming concepts in the future.